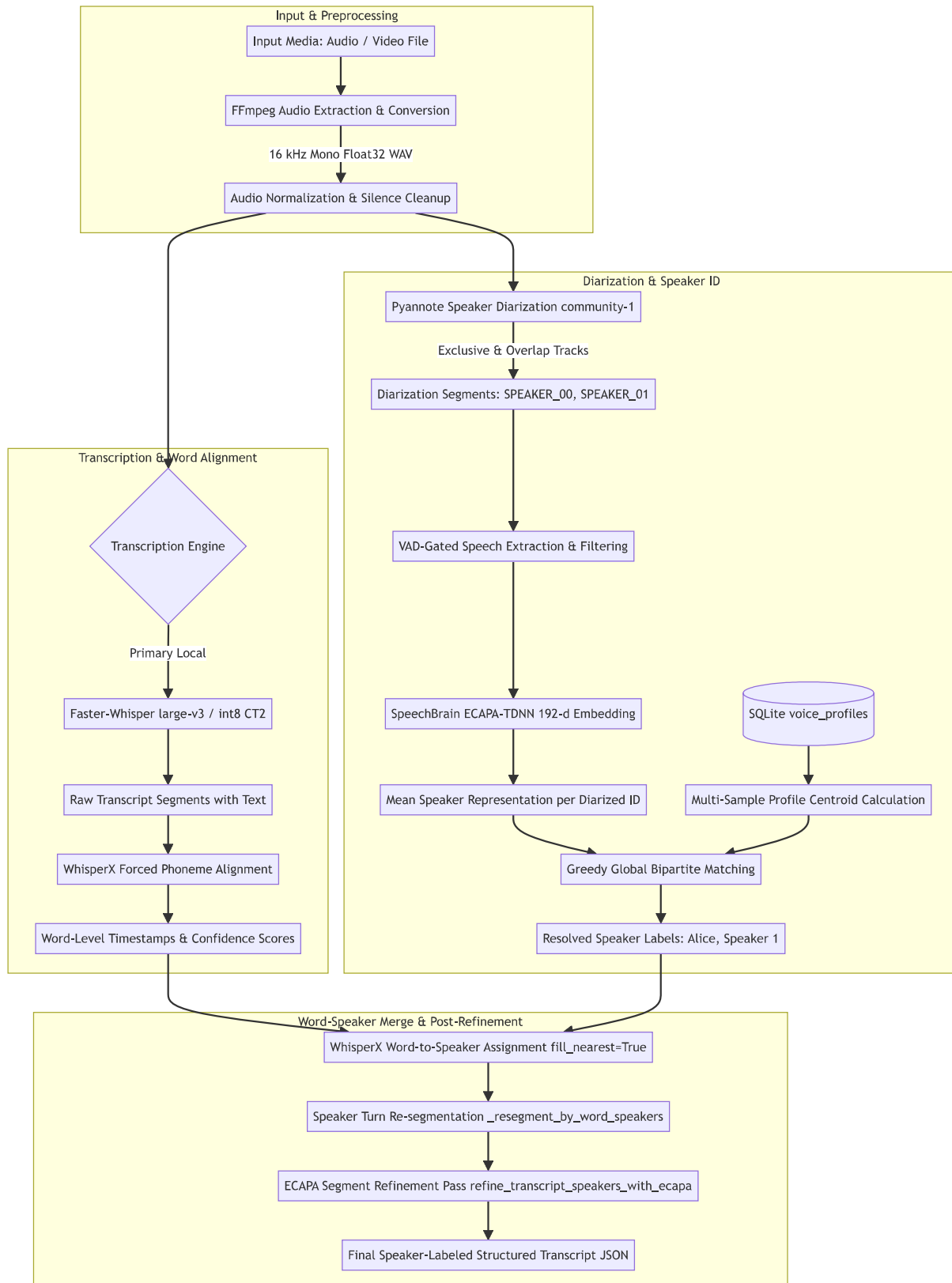


Complete Transcription and Speaker Identification Pipeline



1. Executive Architectural Overview

This document specifies the end-to-end processing pipeline for converting input media (audio or video files) into a fully aligned, high-confidence, speaker-identified transcript. The pipeline is designed for **100% offline, privacy-first processing** (with an optional cloud-based Groq fallback), combining neural acoustic speech recognition, forced phoneme alignment, deep speaker diarization, voice activity detection (VAD), neural speaker embeddings via ECAPA-TDNN, greedy bipartite identity resolution, and segment-level acoustic refinement.

2. Audio Extraction and Preprocessing (FFmpeg)

Implementation Files:

- [backend/utils/audio_utils.py](file:///e:/projects/Transcriber-meeting-initial/backend/utils/audio_utils.py)
- [backend/services/video_processing_service.py](file:///e:/projects/Transcriber-meeting-initial/backend/services/video_processing_service.py)
- [backend/tasks/pipeline.py](file:///e:/projects/Transcriber-meeting-initial/backend/tasks/pipeline.py)

2.1 Media Ingestion & Audio Demuxing

Uploaded files can be any common audio or video format (.mp3 , .wav , .m4a , .mp4 , .mkv , .mov , .webm , .flac , .aac). If a video file is detected, FFmpeg extracts the audio stream without re-encoding the video tracks.

FFmpeg Command Executed:

```
ffmpeg -y -i <input_media_file> -vn -ac 1 -ar 16000 -c:a pcm_s16le
<output_16k_mono.wav>
```

- -vn : Drops video stream entirely.
- -ac 1 : Downmixes multi-channel audio (stereo/5.1) to mono.
- -ar 16000 : Resamples audio to 16 000 Hz (the standard sampling rate expected by Faster-Whisper, WhisperX, Pyannote, and ECAPA-TDNN).
- -c:a pcm_s16le : Encodes to uncompressed 16-bit little-endian PCM.
- On Windows (nt), subprocess execution passes creationflags=0x08000000 (CREATE_NO_WINDOW) to prevent command prompt popups.

2.2 Validation Checks

Before passing the audio to the transcription or diarization engines, `validate_audio()` verifies:

1. **Minimum Duration:** The file must have duration ≥ 1.5 s (configurable via `min_audio_duration_seconds`).
2. **Signal Energy (RMS):** Root-mean-square amplitude must exceed 0.003 (`min_audio_rms_threshold`) to reject silent or corrupted files.

2.3 Audio Conditioning Before Alignment

When `settings.AUDIO_PREPROCESS_BEFORE_ALIGNMENT=True` , a lightweight preprocessing filter is run:

- Silence trimming at the file edges.
- Clipping repair (soft-knee limiting to eliminate digital harshness).
- Loudness normalization to -23 LUFS (EBU R128 standard).

3. Speech-to-Text Transcription

Implementation Files:

- [`backend/services/transcription.py`](file:///e:/projects/Transcriber-meeting-initial/backend/services/transcription.py)
- [`backend/tasks/pipeline.py`](file:///e:/projects/Transcriber-meeting-initial/backend/tasks/pipeline.py)
- [`backend/config.py`](file:///e:/projects/Transcriber-meeting-initial/backend/config.py)

The system supports two execution engines: local **Faster-Whisper** (primary, 100% offline) and **Groq Whisper** (cloud API fallback).

3.1 Primary Engine: Faster-Whisper

- **Model:** OpenAI Whisper `large-v3` , converted and accelerated with CTranslate2.
- **Compute Type:** `int8` (quantized 8-bit weights for reduced VRAM and high throughput) or `float16` when sufficient GPU memory is present.
- **Batch Size:** 8 chunks processed in parallel.
- **Language Detection:** Automatic language detection or user-pinned (e.g., `'en'`).
- **Parallel Subprocess Processing:** When `WHISPER_PARALLEL_PROCESSING > 1` , long audio files are sliced into N -minute segments and processed simultaneously via `ProcessPoolExecutor` , after which memory is freed via explicit garbage collection and CUDA cache flushing (`torch.cuda.empty_cache()`).

```
# Initialization configuration
model = WhisperModel(
    model_size_or_path=model_path,
    device=settings.WHISPER_DEVICE,          # "cuda" | "cpu" | "auto"
    compute_type=settings.WHISPER_COMPUTE_TYPE, # "int8" | "float16"
    download_root=settings.MODELS_DIR,
    local_files_only=True,
)
```

3.3 Stage Output

The transcription output consists of segment dictionaries:

```
{
    "start": 0.52,
    "end": 4.18,
    "text": " Welcome to our quarterly architectural review meeting.",
    "avg_logprob": -0.184,
    "no_speech_prob": 0.002
}
```

4. Word-Level Forced Alignment (WhisperX)

Implementation File: [backend/tasks/pipeline.py](file:///e:/projects/Transcriber-meeting-initial/backend/tasks/pipeline.py)

Raw Whisper segment timestamps are imprecise and frequently drift by 200–800 ms. Precise speaker assignment requires exact word turn boundaries. The system uses **WhisperX** with wav2vec2 forced phoneme alignment.

4.1 Phoneme Alignment Mechanism

1. The language-specific wav2vec2 alignment model is loaded:

```
align_model, align_metadata = whisperx.load_align_model(
    language_code=detected_language,
    device=device,
    model_dir=settings.MODELS_DIR,
)
```

2. `whisperx.align` aligns the audio waveform directly against the transcribed token sequence.
3. Every word is mapped to an exact millisecond interval with an individual acoustic probability score.

4.2 Stage Output Structure

```
{
  "start": 0.52,
  "end": 4.18,
  "text": "Welcome to our quarterly architectural review meeting.",
  "words": [
    { "word": "Welcome",      "start": 0.52, "end": 0.88, "score": 0.941
  },
    { "word": "to",          "start": 0.90, "end": 0.98, "score": 0.882
  },
    { "word": "our",         "start": 1.00, "end": 1.12, "score": 0.915
  },
    { "word": "quarterly",   "start": 1.14, "end": 1.72, "score": 0.963
  },
    { "word": "architectural", "start": 1.76, "end": 2.64, "score": 0.957
  },
    { "word": "review",      "start": 2.68, "end": 3.10, "score": 0.981
  },
    { "word": "meeting.",    "start": 3.14, "end": 3.78, "score": 0.974
  }
  ]
}
```

5. Speaker Diarization (Pyannote)

Implementation File: [`backend/services/diarization.py`](file:///e:/projects/Transcriber-meeting-initial/backend/services/diarization.py)

Speaker diarization answers the question "*who spoke when*" in terms of raw abstract acoustic clusters (`SPEAKER_00` , `SPEAKER_01`).

5.1 Pyannote Community-1 Architecture

The system runs `pyannote/speaker-diarization-community-1` fully offline from local snapshot files in `MODELS_DIR/audio_context/` :

- segmentation/pytorch_model.bin : Neural acoustic voice turn segmentation.
- embedding/pytorch_model.bin : CAM++ acoustic representation model.
- plda/plda.npz and plda/xvec_transform.npz : Variational Bayes (VBx) clustering with Probabilistic Linear Discriminant Analysis.

5.2 Dual Annotation & Overlap Recovery

Pyannote 4.x generates a composite `DiarizeOutput` containing two separate timeline tracks:

Annotation Track	Purpose	Usage in Engine
exclusive_speaker_diarization	Exactly one active speaker per timestamp	Primary timeline for word speaker assignment (avoids conflicting turns)
speaker_diarization	Multi-speaker tracking with simultaneous speech	Used strictly for overlap detection

5.3 Overlap Detection & Merging Algorithm

1. The regular annotation is examined for time intersections between distinct speakers:

$$\Delta t = \min(t_{\text{end},A}, t_{\text{end},B}) - \max(t_{\text{start},A}, t_{\text{start},B}) > 0.10 \text{ s} \quad (100 \text{ ms})$$

2. Unique overlap windows are collected: `{"start": float, "end": float, "speakers": ["SPEAKER_00", "SPEAKER_01"]}`.
3. Every segment in the exclusive track that intersects an overlap window by $> 100 \text{ ms}$ is tagged with:
 - `is_overlap = True`
 - `overlap_regions = [{"start": s, "end": e, "speakers": [...]}]`

5.4 Fallback: Energy-Based Diarization

If Pyannote weights are unavailable, the engine falls back to `_diarize_energy`:

- Calculates RMS energy via Librosa (2048 frame, 512 hop).
- Determines voice activity using the 30th percentile energy threshold.
- Alternates speaker labels across silence gaps $\geq 0.8 \text{ s}$.
- Filters out turns shorter than `settings.MIN_SEGMENT_DURATION` (1.5 s).

6. ECAPA-TDNN Speaker Embeddings

Implementation File: [backend/services/embedding.py](file:///e:/projects/Transcriber-meeting-initial/backend/services/embedding.py)

SpeechBrain's ECAPA-TDNN (*Emphasized Channel Attention, Propagation and Aggregation Time Delay Neural Network*) transforms variable-length voiced speech into a fixed-length identity representation.

6.1 Specifications

- **Model:** speechbrain/spkrec-ecapa-voxceleb
- **Embedding Dimension:** $d = 192$ float32 values
- **Offline Inference:** `FetchConfig(allow_network=False)` with local HuggingFace cache seeding.
- **Normalization:** Every extracted vector is strictly L_2 -normalized:

$$\mathbf{e} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}, \quad \|\mathbf{e}\|_2 = 1.0$$

6.2 VAD-Gated Extraction (`vad_extract_speaker_embedding`)

Raw diarized audio segments often contain dead silence, breaths, or table thumps. Feeding unvoiced audio degrades embedding quality. The extraction pipeline applies:

1. **Frame-level VAD:** `webrtcvad` (aggressiveness 2, 30 ms frames) with fallback to energy quantile (0.25).
2. **Filtering & Merging:** Removes speech chunks < 0.25 s and bridges pauses < 0.40 s.
3. **Speech Duration Conditions:**
 - < 2.5 s (`min_speech_sec`): Discarded as insufficient data. Returns `None`.
 - $2.5 \text{ s} \leq t \leq 8.0 \text{ s}$ (`max_speech_sec`): Processed in a single forward pass.
 - > 8.0 s (**Long Speech**): Sliced into sliding windows of 4.0 s (`win_sec`) with 3.0 s hop (`hop_sec` , 1.0 s overlap). All window embeddings are averaged and re-normalized:

$$\hat{\mathbf{e}} = \frac{\frac{1}{K} \sum_{k=1}^K \mathbf{e}_k}{\left\| \frac{1}{K} \sum_{k=1}^K \mathbf{e}_k \right\|_2}$$

7. Voice Profile Enrollment and Centroids

Implementation Files:

- [backend/database.py](file:///e:/projects/Transcriber-meeting-initial/backend/database.py)

- [backend/routers/voice.py](file:///e:/projects/Transcriber-meeting-initial/backend/routers/voice.py)

7.1 Database Schema

Voice profiles are stored in SQLite in the `voice_profiles` table:

```
CREATE TABLE IF NOT EXISTS voice_profiles (
    id TEXT PRIMARY KEY,          -- UUID string
    user_id TEXT NOT NULL,        -- Owning user ID
    label TEXT NOT NULL,          -- Display name (e.g., "John
Doe")
    embeddings TEXT NOT NULL DEFAULT '[]', -- JSON array of 192-d
vectors: List[List[float]]
    sample_count INTEGER NOT NULL DEFAULT 0,
    is_self INTEGER NOT NULL DEFAULT 0,    -- 1 if user's personal
profile, 0 for colleagues
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL
);
```

7.2 Enrollment Mechanisms

1. **User Onboarding** (`/voice/sample` + `/voice/finalize-setup`): Records 1–3 clean audio samples (≥ 2.0 s, $\text{RMS} \geq 0.003$). Stored as the primary profile (`is_self = 1`).
2. **Team Member Addition** (`/voice/add-profile`): Uploads audio files for colleagues, storing their 192-d embeddings under distinct labels.
3. **Bulk Folder Ingestion** (`/voice/bulk-folder-import`): Recursively parses folders structured as `Root/<SpeakerName>/sample.wav` . Merges new 192-d embeddings into existing profiles and rejects incompatible dimensions (e.g., legacy 256-d or 512-d vectors).
4. **In-Meeting Transcript Training** (`/voice/extract-samples` + `/voice/train-from-transcript`):
 - Automatically searches past meeting transcripts for candidate segments where the speaker spoke ≥ 2.0 s without overlap.
 - Sorts candidates by Whisper's `avg_logprob` (highest acoustic clarity).
 - Uses FFmpeg to extract 3–5 audio slices.
 - Computes ECAPA embeddings and updates the profile in the database.
 - Automatically invokes `sync_global_speaker_rename()` to update historical transcripts, meeting summaries, and Record of Meeting (ROM) documents.

7.3 Profile Centroid Calculation

To avoid noisy matches from single outlier samples, every voice profile is represented by a **multi-sample centroid**:

- **Single Sample:** $\mathbf{c}_p = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|_2}$ (`scoring_method = "single_sample"`).
- **Multiple Samples** ($M > 1$):

$$\mathbf{c}_p = \frac{\sum_{j=1}^M \mathbf{v}_j}{\left\| \sum_{j=1}^M \mathbf{v}_j \right\|_2} \quad (\text{scoring_method} = \text{"centroid"})$$

- **Fallback:** If centroid calculation fails, `best_match_similarity()` computes $\max_j \cos(\mathbf{e}, \mathbf{v}_j)$ (`scoring_method = "sample_max_fallback"`).

8. Speaker Matching Logic & Thresholds

Implementation File: [`backend/services/identification.py`](file:///e:/projects/Transcriber-meeting-initial/backend/services/identification.py)

8.1 Speaker-Level Identification

Matching is performed at the **speaker level**, rather than per-segment, to prevent label flipping across adjacent turns:

1. All non-overlap diarization segments ≥ 0.5 s are grouped by raw ID (`SPEAKER_00` , `SPEAKER_01` , ...).
2. For each raw ID, segment embeddings are extracted and averaged to create a single representative speaker vector $\mathbf{e}_{\text{spk}}$.

8.2 Cosine Similarity Metric

Similarity between a detected speaker $\mathbf{u}$ and profile centroid $\mathbf{v}$ is given by:

$$S_C(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \mathbf{u} \cdot \mathbf{v} \quad (\text{since } \|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1.0)$$

8.3 Greedy Global Bipartite Matching

To resolve matches globally and prevent two diarization clusters from claiming the same enrolled person:

1. Compute similarity matrix $S_{i,j} = S_C(\mathbf{e}_{\text{spk},i}, \mathbf{c}_{p,j})$ for all pairs.
2. Pool all candidate pairs into a list and sort in **descending order of similarity**.
3. Iterate greedily through candidates:

- If neither speaker cluster i nor profile j has been assigned, and $S_{i,j} \geq \text{threshold}$:
 - Assign profile j to speaker cluster i .
 - Lock both i and j from further matching.
- 4. If a speaker cluster remains unassigned after checking all candidates:
 - It is assigned a generic label: Speaker 1 , Speaker 2 , etc.
 - Its highest observed similarity is retained with `scoring_method = "no_match"` .

8.4 Effective Threshold Resolution

- **Legacy Threshold:** 0.75 (Resemblyzer baseline).
- **ECAPA-TDNN Threshold:** 0.72 (`settings.SPEAKER_SIMILARITY_THRESHOLD_ECAPA_TDNN`).
- **Auto-Substitution:** If the caller provides 0.75, the engine overrides it to 0.72 because ECAPA-TDNN scores are calibrated differently.

9. Word-Speaker Assignment and Re-segmentation

Implementation Files:

- [`backend/tasks/pipeline.py`](file:///e:/projects/Transcriber-meeting-initial/backend/tasks/pipeline.py)
- [`backend/services/identification.py`](file:///e:/projects/Transcriber-meeting-initial/backend/services/identification.py)

9.1 WhisperX Word-Level Speaker Assignment

The identified diarization segments are formatted into a DataFrame and passed to WhisperX:

```
wx_assigned = whisperx.assign_word_speakers(
    diar_df,
    aligned_result,
    fill_nearest=True
)
```

- `fill_nearest=True` : Words falling into silence gaps or slight turn boundary offsets are assigned to the nearest speaker rather than left unassigned.
- Fallback: `_assign_speakers_to_words_manual` handles assignment via maximum temporal overlap if WhisperX raises an exception.

9.2 Turn Re-segmentation (`_resegment_by_word_speakers`)

Whisper segments often span multiple speakers if a speaker change occurred mid-sentence. The re-segmentation algorithm splits segments at exact speaker change points:

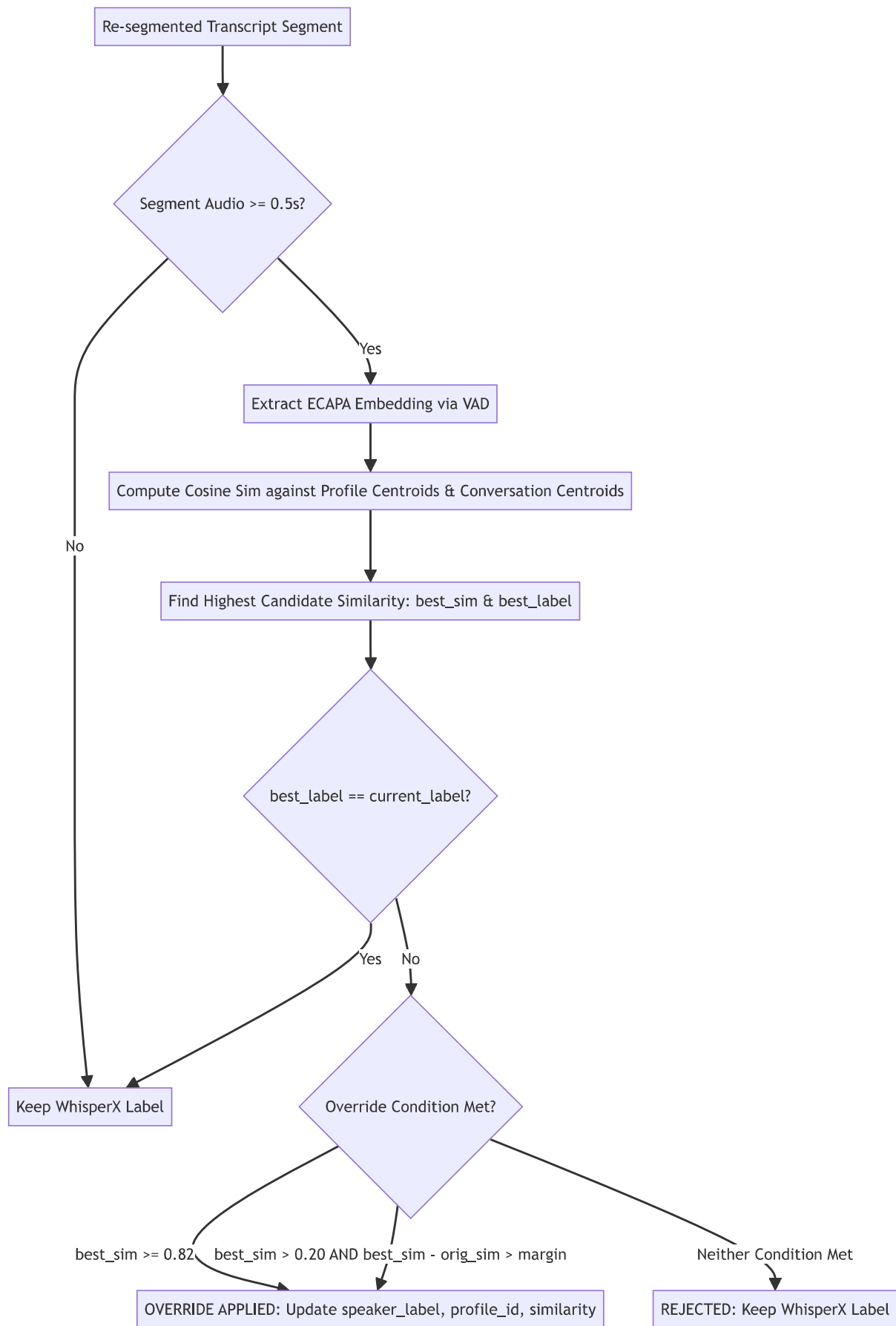
- Iterates word by word.
- Whenever `word["speaker"]` changes, the current segment is closed and a new segment is initialized.
- Guarantees that every resulting transcript segment contains speech from **only one speaker**.

10. Post-Alignment ECAPA Refinement Pass

Implementation File: [`backend/services/identification.py:498`](file:///e:/projects/Transcriber-meeting-initial/backend/services/identification.py#L498)

(`refine_transcript_speakers_with_ecapa`)

Acoustic boundary errors during forced alignment can cause short segments to be misattributed. The post-alignment refinement pass validates and corrects segments using localized audio embeddings:



10.1 Override Logic

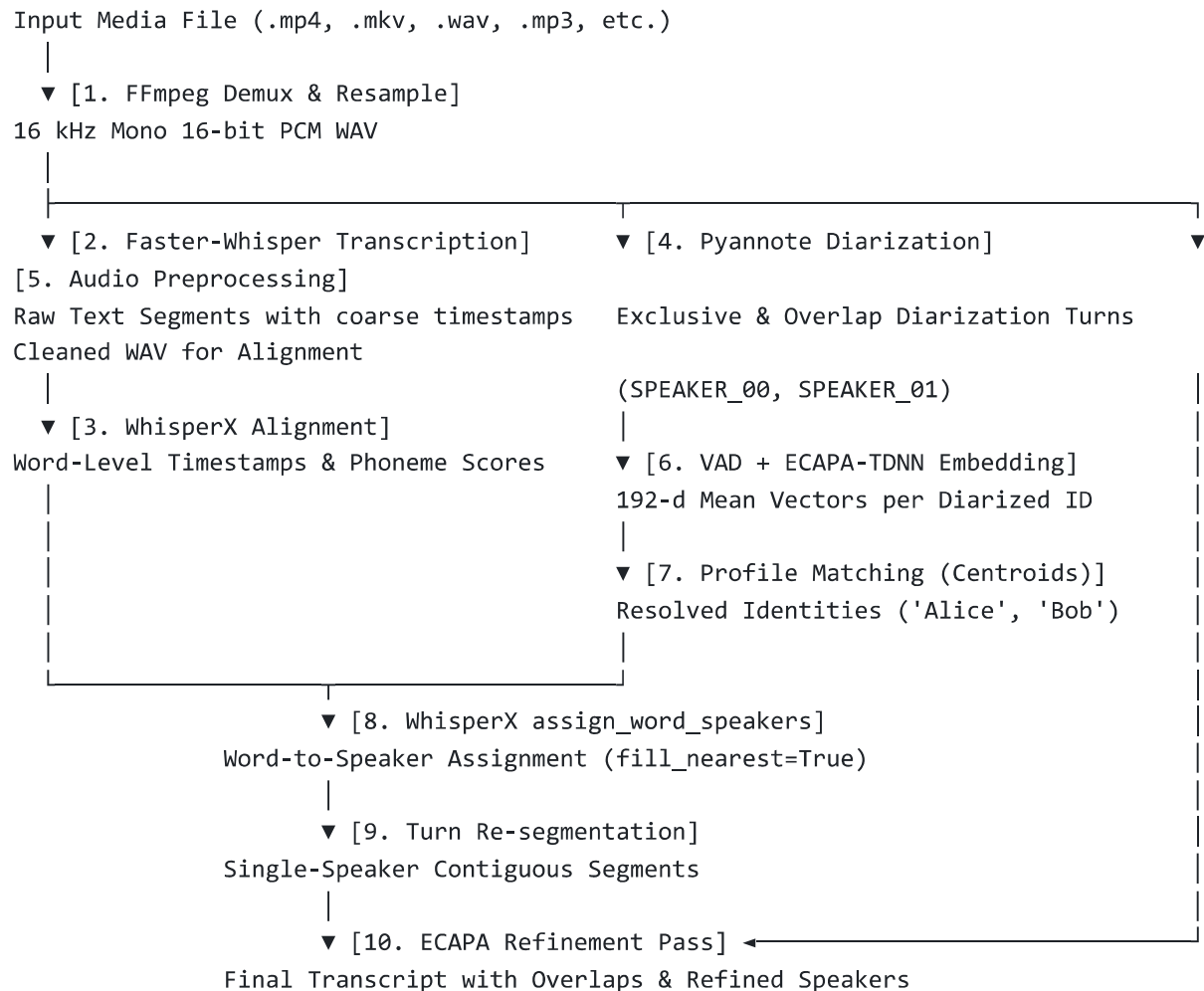
For each segment s , compare its embedding against enrolled profiles and in-conversation speaker centroids:

- Let orig_sim be the segment's similarity to its currently assigned speaker.
- Let best_sim be the highest similarity to any alternative speaker.
- The label is overridden if and only if:

$$\text{best_sim} \geq 0.82 \quad \vee \quad \left(\text{best_sim} > 0.20 \wedge (\text{best_sim} - \text{orig_sim}) > \text{margin} \right)$$

(where default $\text{margin} = 0.30$, configured via `settings.speaker_refinement_margin`).

11. Complete Stage-by-Stage Input/Output Specification



Final Transcript Schema (JSON)

```
[
  {
    "start": 0.52,
    "end": 3.78,
    "speaker": "SPEAKER_00",
    "speaker_label": "Alice Smith",
    "speaker_profile_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "similarity": 0.8842,
    "scoring_method": "centroid",
    "is_overlap": false,
    "overlap_regions": [],
    "text": "Welcome to our quarterly architectural review meeting.",
    "words": [
      {
        "word": "Welcome",
        "start": 0.52,
        "end": 0.88,
        "score": 0.941,
        "speaker_label": "Alice Smith"
      },
      {
        "word": "meeting.",
        "start": 3.14,
        "end": 3.78,
        "score": 0.974,
        "speaker_label": "Alice Smith"
      }
    ]
  },
  {
    "start": 3.90,
    "end": 6.45,
    "speaker": "SPEAKER_01",
    "speaker_label": "Speaker 2",
    "speaker_profile_id": null,
    "similarity": 0.4120,
    "scoring_method": "no_match",
    "is_overlap": true,
    "overlap_regions": [
      {
        "start": 5.80,
```

```
    "end": 6.45,  
    "speakers": ["Alice Smith", "Speaker 2"]  
  }  
],  
  "text": "Thanks Alice. Let's start with the database performance.",  
  "words": [ ... ]  
}  
]
```